Laboratório Aula 16

November 3, 2026

1 Roteiro de Laboratório Prático

1.1 Etapa Preliminar: Importação das Bibliotecas

[1]:
```
import time
from itertools import combinations
from collections import defaultdict
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons, make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.neighbors import NearestNeighbors
from scipy.cluster.hierarchy import linkage, dendrogram
```

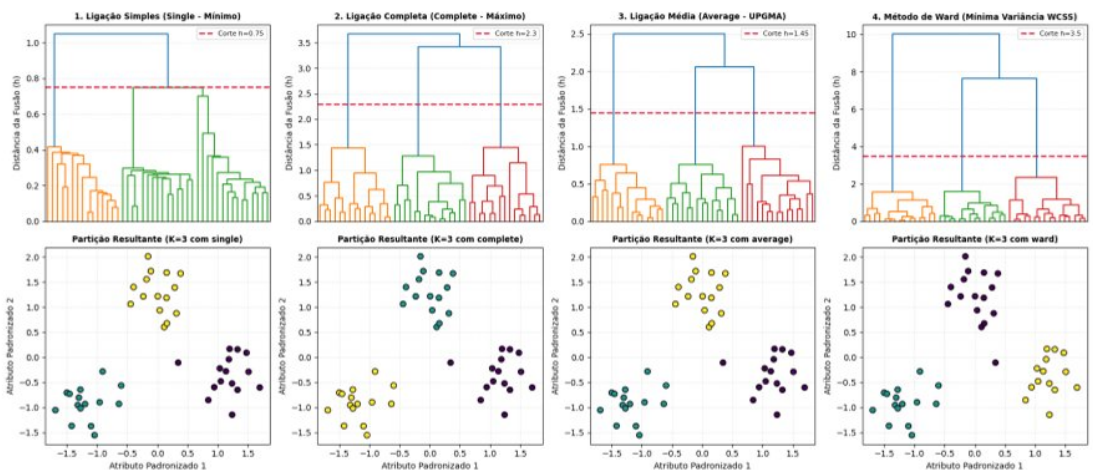
1.2 Experimento 1: Agrupamento Hierárquico – O Efeito dos Critérios de Ligação (*Single*, *Complete*, *Average* e *Ward*) e o Corte do Dendrograma

[2]:
```
np.random.seed(42)
X_hier, _ = make_blobs(n_samples=45, centers=[[-3, -2], [0, 3], [3, -1]],␣
 ↪cluster_std=0.85, random_state=42)
X_hier_norm = StandardScaler().fit_transform(X_hier)

metodos_ligacao = [
    ('single', '1. Ligação Simples (Single - Mínimo)', 0.75),
    ('complete', '2. Ligação Completa (Complete - Máximo)', 2.30),
    ('average', '3. Ligação Média (Average - UPGMA)', 1.45),
    ('ward', '4. Método de Ward (Mínima Variância WCSS)', 3.50)
]

fig, eixos = plt.subplots(2, 4, figsize=(18, 7.8))
for col, (metodo, titulo, altura_corte) in enumerate(metodos_ligacao):
    Z_mat = linkage(X_hier_norm, method=metodo, metric='euclidean')
    ax_dend = eixos[0, col]
    dendrogram(Z_mat, ax=ax_dend, color_threshold=altura_corte, no_labels=True)
    ax_dend.axhline(y=altura_corte, color='crimson', linestyle='--',␣
 ↪linewidth=1.8, label=f'Corte h={altura_corte}')
```

```
    ax_dend.set_title(titulo, fontweight='bold', fontsize=9.5)
    ax_dend.set_ylabel("Distância da Fusão (h)")
    ax_dend.legend(loc='upper right', fontsize=8)
    ax_dend.grid(True, axis='y', linestyle=':', alpha=0.5)
    modelo_aglom = AgglomerativeClustering(n_clusters=3, linkage=metodo)
    rotulos_k3 = modelo_aglom.fit_predict(X_hier_norm)
    ax_scat = eixos[1, col]
    ax_scat.scatter(X_hier_norm[:, 0], X_hier_norm[:, 1], c=rotulos_k3,␣
 ↪cmap='viridis', s=45, edgecolor='k')
    ax_scat.set_title(f"Partição Resultante (K=3 com {metodo})",␣
 ↪fontweight='bold', fontsize=9.5)
    ax_scat.set_xlabel("Atributo Padronizado 1")
    ax_scat.set_ylabel("Atributo Padronizado 2")
    ax_scat.grid(True, linestyle=':', alpha=0.5)
plt.tight_layout()
plt.show()
```



1.3 Experimento 2: Agrupamento por Densidade (DBSCAN) – Anatomia dos Pontos (Central, Borda e Ruído -1) e Curva *k-distance*

[3]:
```
X_luas, _ = make_moons(n_samples=220, noise=0.06, random_state=42)
np.random.seed(42)
outliers_luas = np.array([[-1.5, 1.3], [2.4, 1.4], [0.5, 1.7], [-0.8, -1.1], [1.
 ↪8, -1.2], [2.5, -0.5]])
X_luas_ruido = np.vstack([X_luas, outliers_luas])
X_luas_norm = StandardScaler().fit_transform(X_luas_ruido)
min_pts = 5
vizinhos = NearestNeighbors(n_neighbors=min_pts).fit(X_luas_norm)
distancias_viz, _ = vizinhos.kneighbors(X_luas_norm)
k_dist_ordenada = np.sort(distancias_viz[:, min_pts - 1])
```

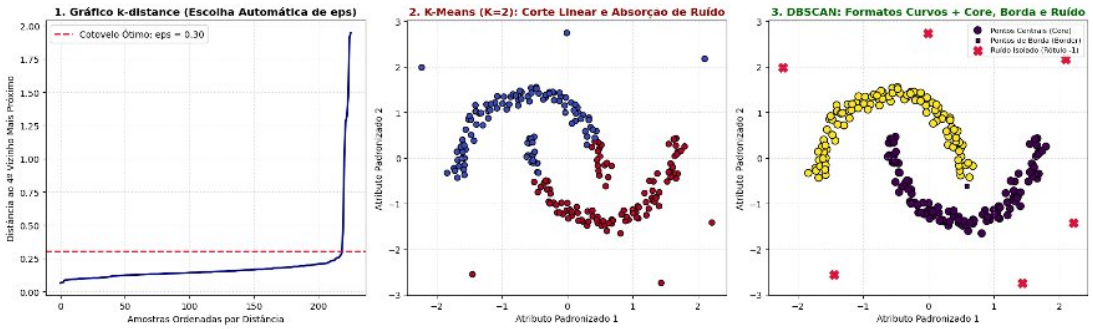
```
rotulos_kmeans = KMeans(n_clusters=2, n_init=10, random_state=42).
 ↪fit_predict(X_luas_norm)
modelo_dbscan = DBSCAN(eps=0.30, min_samples=min_pts).fit(X_luas_norm)
rotulos_db = modelo_dbscan.labels_
mascara_core = np.zeros_like(rotulos_db, dtype=bool)
mascara_core[modelo_dbscan.core_sample_indices_] = True
mascara_ruido = (rotulos_db == -1)
mascara_borda = (~mascara_core) & (~mascara_ruido)
fig, (ax_kdist, ax_km, ax_db) = plt.subplots(1, 3, figsize=(17, 5.2))

ax_kdist.plot(k_dist_ordenada, color='navy', linewidth=2.2)
ax_kdist.axhline(y=0.30, color='crimson', linestyle='--', linewidth=1.8,␣
 ↪label='Cotovelo Ótimo: eps = 0.30')
ax_kdist.set_title("1. Gráfico k-distance (Escolha Automática de eps)",␣
 ↪fontweight='bold')
ax_kdist.set_xlabel("Amostras Ordenadas por Distância")
ax_kdist.set_ylabel(f"Distância ao {min_pts-1}º Vizinho Mais Próximo")
ax_kdist.legend(loc='upper left')
ax_kdist.grid(True, linestyle=':', alpha=0.6)

ax_km.scatter(X_luas_norm[:, 0], X_luas_norm[:, 1], c=rotulos_kmeans,␣
 ↪cmap='coolwarm', s=40, edgecolor='k')
ax_km.set_title("2. K-Means (K=2): Corte Linear e Absorção de Ruído",␣
 ↪color='darkred', fontweight='bold')
ax_km.set_xlabel("Atributo Padronizado 1")
ax_km.set_ylabel("Atributo Padronizado 2")
ax_km.grid(True, linestyle=':', alpha=0.6)

ax_db.scatter(X_luas_norm[mascara_core, 0], X_luas_norm[mascara_core, 1],
              c=rotulos_db[mascara_core], cmap='viridis', s=65, edgecolor='k',␣
 ↪label='Pontos Centrais (Core)')
ax_db.scatter(X_luas_norm[mascara_borda, 0], X_luas_norm[mascara_borda, 1],
              c=rotulos_db[mascara_borda], cmap='viridis', s=25, marker='s',␣
 ↪edgecolor='k', label='Pontos de Borda (Border)')
ax_db.scatter(X_luas_norm[mascara_ruido, 0], X_luas_norm[mascara_ruido, 1],
              c='crimson', s=90, marker='X', lw=1.5, label='Ruído Isolado␣
 ↪(Rótulo -1)')
ax_db.set_title("3. DBSCAN: Formatos Curvos + Core, Borda e Ruído",␣
 ↪color='darkgreen', fontweight='bold')
ax_db.set_xlabel("Atributo Padronizado 1")
ax_db.set_ylabel("Atributo Padronizado 2")
ax_db.legend(loc='upper right', fontsize=8)
ax_db.grid(True, linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()
```



1.4 Experimento 3: Abrindo a Caixa-Preta do Apriori – Geração de Candidatos (C_k), Poda Antimonotônica (L_k) e Visualização da Árvore de Busca

[8]:
```
transacoes_loja = [
    {'Cerveja', 'Fralda', 'Goma', 'Refri'},
    {'Cerveja', 'Fralda'},
    {'Cerveja', 'Fralda', 'Goma', 'Lanche'},
    {'Cerveja', 'Fralda', 'Refri', 'Lanche'},
    {'Fralda', 'Refri'},
    {'Fralda'},
    {'Fralda'},
    {'Refri', 'Lanche'},
    {'Refri', 'Lanche'}
]

min_sup_loja = 0.40
N_trans = len(transacoes_loja)
def apriori_transparente(transacoes, min_sup):
    itens_unicos = sorted(list(set.union(*transacoes)))
    historico_niveis = {}
    k = 1
    C_k = [(item,) for item in itens_unicos]
    while C_k:
        contagens_Ck = {}
        L_k = {}
        for cand in C_k:
            conjunto_cand = set(cand)
            contagem = sum(1 for t in transacoes if conjunto_cand.issubset(t))
            suporte = contagem / len(transacoes)
            contagens_Ck[cand] = suporte
            if suporte >= min_sup:
                L_k[cand] = suporte
        historico_niveis[k] = {'C_k': contagens_Ck, 'L_k': L_k}
```

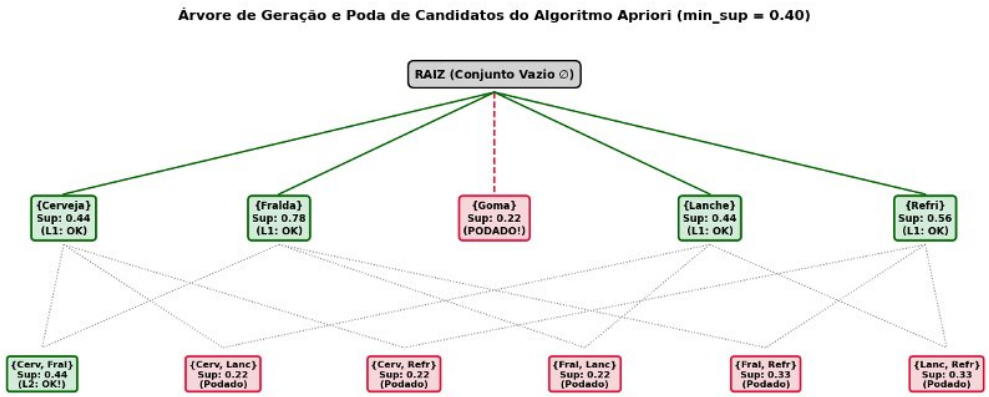
```
        print(f"=== NÍVEL k = {k} | Candidatos Testados (|C_{k}| = {len(C_k)})␣
↪-> Aprovados (|L_{k}| = {len(L_k)}) ===")
        for cand, sup in contagens_Ck.items():
            status = "APROVADO (L_" + str(k) + ")" if cand in L_k else "PODADO␣
↪(< min_sup)"
            print(f"  Itemset {set(cand)}: Suporte = {sup:.4f} -> {status}")
        if len(L_k) < 2:
            print(f"\n[PARADA DO APRIORI]: Não é possível formar candidatos de␣
↪tamanho k = {k+1}!\n")
            break
        itens_sobreviventes = sorted(list(set(item for tupla in L_k.keys() for␣
↪item in tupla)))
        novos_candidatos = []
        for comb in combinations(itens_sobreviventes, k + 1):
            if all(sub in L_k for sub in combinations(comb, k)):
                novos_candidatos.append(comb)
        C_k = novos_candidatos
        k += 1
    return historico_niveis
historico_apriori = apriori_transparente(transacoes_loja, min_sup_loja)
fig, ax = plt.subplots(figsize=(13, 5.2))
ax.set_title("Árvore de Geração e Poda de Candidatos do Algoritmo Apriori␣
↪(min_sup = 0.40)", fontweight='bold', fontsize=12)
ax.text(0.5, 2.8, "RAIZ (Conjunto Vazio $\emptyset$)", ha='center',␣
↪va='center', fontweight='bold',
        bbox=dict(boxstyle='round,pad=0.5', facecolor='lightgray',␣
↪edgecolor='black', lw=1.5))
c1_dict = historico_apriori[1]['C_k']
l1_dict = historico_apriori[1]['L_k']
pos_x_k1 = np.linspace(0.1, 0.9, len(c1_dict))
mapa_pos_k1 = {}
for idx, (cand, sup) in enumerate(c1_dict.items()):
    x = pos_x_k1[idx]
    mapa_pos_k1[cand[0]] = (x, 1.6)
    aprovado = cand in l1_dict
    cor_fundo = '#d4edda' if aprovado else '#f8d7da'
    cor_borda = 'darkgreen' if aprovado else 'crimson'
    estilo_linha = '-' if aprovado else '--'
    ax.plot([0.5, x], [2.65, 1.8], color=cor_borda, linestyle=estilo_linha,␣
↪lw=1.5)
    rotulo_no = f"{{{cand[0]}}}\nSup: {sup:.2f}\n({'L1: OK' if aprovado else␣
↪'PODADO!'})"
    ax.text(x, 1.6, rotulo_no, ha='center', va='center', fontsize=8.5,␣
↪fontweight='bold',
            bbox=dict(boxstyle='round,pad=0.4', facecolor=cor_fundo,␣
↪edgecolor=cor_borda, lw=1.8))
```

```
c2_dict = historico_apriori[2]['C_k']
l2_dict = historico_apriori[2]['L_k']
pos_x_k2 = np.linspace(0.08, 0.92, len(c2_dict))

for idx, (cand, sup) in enumerate(c2_dict.items()):
    x = pos_x_k2[idx]
    aprovado = cand in l2_dict
    cor_fundo = '#d4edda' if aprovado else '#f8d7da'
    cor_borda = 'darkgreen' if aprovado else 'crimson'
    for item_pai in cand:
        xp, yp = mapa_pos_k1[item_pai]
        ax.plot([xp, x], [yp - 0.22, 0.52], color='gray', linestyle=':', lw=1.0)
    rotulo_no = f"{{{cand[0][:4]}, {cand[1][:4]}}}\nSup: {sup:.2f}\n({'L2: OK!' if aprovado else 'Podado'})"
    ax.text(x, 0.3, rotulo_no, ha='center', va='center', fontsize=8, fontweight='bold',
            bbox=dict(boxstyle='round,pad=0.35', facecolor=cor_fundo, edgecolor=cor_borda, lw=1.8))
ax.set_xlim(0, 1)
ax.set_ylim(-0.1, 3.2)
ax.axis('off')
plt.tight_layout()
plt.show()
```

```
=== NÍVEL k = 1 | Candidatos Testados (|C_1| = 5) -> Aprovados (|L_1| = 4) ===
  Itemset {'Cerveja'}: Suporte = 0.4444 -> APROVADO (L_1)
  Itemset {'Fralda'}: Suporte = 0.7778 -> APROVADO (L_1)
  Itemset {'Goma'}: Suporte = 0.2222 -> PODADO (< min_sup)
  Itemset {'Lanche'}: Suporte = 0.4444 -> APROVADO (L_1)
  Itemset {'Refri'}: Suporte = 0.5556 -> APROVADO (L_1)
=== NÍVEL k = 2 | Candidatos Testados (|C_2| = 6) -> Aprovados (|L_2| = 1) ===
  Itemset {'Fralda', 'Cerveja'}: Suporte = 0.4444 -> APROVADO (L_2)
  Itemset {'Lanche', 'Cerveja'}: Suporte = 0.2222 -> PODADO (< min_sup)
  Itemset {'Refri', 'Cerveja'}: Suporte = 0.2222 -> PODADO (< min_sup)
  Itemset {'Lanche', 'Fralda'}: Suporte = 0.2222 -> PODADO (< min_sup)
  Itemset {'Refri', 'Fralda'}: Suporte = 0.3333 -> PODADO (< min_sup)
  Itemset {'Refri', 'Lanche'}: Suporte = 0.3333 -> PODADO (< min_sup)

[PARADA DO APRIORI]: Não é possível formar candidatos de tamanho k = 3!
```



1.5 Experimento 4: Construindo e Visualizando a Árvore FP-Tree do Zero (Nó a Nó) + Base de Padrões Condicionais

[9]:
```
class NoFPTree:
    def __init__(self, item, contagem=1, pai=None):
        self.item = item
        self.contagem = contagem
        self.pai = pai
        self.filhos = {}
def construir_fptree(transacoes, min_contagem):
    # ETAPA 1: 1ª Varredura para contar a frequência individual de cada item
    freq_individual = defaultdict(int)
    for t in transacoes:
        for item in set(t):
            freq_individual[item] += 1
    itens_frequentes = {k: v for k, v in freq_individual.items() if v >=␣
 ↪min_contagem}
    ordem_L = sorted(itens_frequentes.keys(), key=lambda x:␣
 ↪(-itens_frequentes[x], x))
    print("1. Lista Ordenada de Itens Frequentes (L):", [(i,␣
 ↪itens_frequentes[i]) for i in ordem_L])
    raiz = NoFPTree("NULL", contagem=0, pai=None)
    tabela_cabecalho = {item: [] for item in ordem_L}
    print("\n2. Transações Filtradas e Reordenadas Inseridas na FP-Tree:")
    for idx, t in enumerate(transacoes, 1):
        trans_ordenada = [item for item in ordem_L if item in t]
        print(f"   T{idx}: Bruta={t} -> Ordenada={trans_ordenada}")
        no_atual = raiz
        for item in trans_ordenada:
            if item in no_atual.filhos:
                no_atual.filhos[item].contagem += 1
```

```
            else:
                novo_no = NoFPTree(item, contagem=1, pai=no_atual)
                no_atual.filhos[item] = novo_no
                tabela_cabecalho[item].append(novo_no)
            no_atual = no_atual.filhos[item]
    return raiz, tabela_cabecalho, ordem_L
def minerar_padroes_condicionais(tabela_cabecalho, ordem_L, min_contagem):
    print("\n3. Mineração Bottom-Up (Base de Padrões Condicionais e Árvore FP␣
 ↪Condicional):")
    for item_sufixo in reversed(ordem_L):
        caminhos_prefixo = []
        contagem_acumulada_prefixos = defaultdict(int)
        for no in tabela_cabecalho[item_sufixo]:
            contagem_folha = no.contagem
            caminho = []
            cursor_pai = no.pai
            while cursor_pai is not None and cursor_pai.item != "NULL":
                caminho.append(cursor_pai.item)
                contagem_acumulada_prefixos[cursor_pai.item] += contagem_folha
                cursor_pai = cursor_pai.pai
            if caminho:
                caminhos_prefixo.append((list(reversed(caminho)),␣
 ↪contagem_folha))
        arvore_condicional = {k: v for k, v in contagem_acumulada_prefixos.
 ↪items() if v >= min_contagem}
        padroes_gerados = [({k, item_sufixo}, v) for k, v in arvore_condicional.
 ↪items()]
        print(f"   Item '{item_sufixo}': Base Condicional = {caminhos_prefixo}")
        print(f"            -> Árvore FP Condicional (>= {min_contagem}) =␣
 ↪{arvore_condicional} | Pares = {padroes_gerados}")
transacoes_fptree = [
    ['E', 'K', 'M', 'N', 'O', 'Y'],
    ['D', 'E', 'K', 'N', 'O', 'Y'],
    ['A', 'E', 'K', 'M'],
    ['C', 'K', 'M', 'U', 'Y'],
    ['C', 'E', 'I', 'K', 'O']
]
raiz_fp, cabecalho_fp, lista_L = construir_fptree(transacoes_fptree,␣
 ↪min_contagem=3)
minerar_padroes_condicionais(cabecalho_fp, lista_L, min_contagem=3)

def desenhar_fptree_matplotlib(raiz):
    fig, ax = plt.subplots(figsize=(10, 6.2))
    ax.set_title("Estrutura Visual da Árvore FP-Tree Construída na Memória (T1␣
 ↪a T5, min_sup = 3)",
                 fontweight='bold', fontsize=11.5)
```

```
    posicoes = {}
    contador_folhas = [0]
    def calcular_layout(no, profundidade=0):
        if not no.filhos:
            x = contador_folhas[0] * 1.5
            contador_folhas[0] += 1
        else:
            xs_filhos = [calcular_layout(f, profundidade + 1) for f in no.
 ↪filhos.values()]
            x = float(np.mean(xs_filhos))
        y = -profundidade * 1.2
        posicoes[no] = (x, y)
        return x

    calcular_layout(raiz)
    for no, (x, y) in posicoes.items():
        for filho in no.filhos.values():
            xf, yf = posicoes[filho]
            ax.plot([x, xf], [y, yf], color='slategray', lw=2.0, zorder=1)

        eh_raiz = (no.item == "NULL")
        cor_no = 'lightgray' if eh_raiz else ('#c3e6cb' if no.contagem >= 3␣
 ↪else '#d1ecf1')
        borda_no = 'black' if eh_raiz else ('darkgreen' if no.contagem >= 3␣
 ↪else 'steelblue')
        texto_no = "NULL\n(Raiz)" if eh_raiz else f"{no.item} : {no.contagem}"
        ax.text(x, y, texto_no, ha='center', va='center', fontweight='bold',␣
 ↪fontsize=9.5, zorder=3,
                bbox=dict(boxstyle='circle,pad=0.45' if not eh_raiz else␣
 ↪'round,pad=0.4',
                          facecolor=cor_no, edgecolor=borda_no, lw=2.0))

    ax.axis('off')
    plt.tight_layout()
    plt.show()
desenhar_fptree_matplotlib(raiz_fp)
```

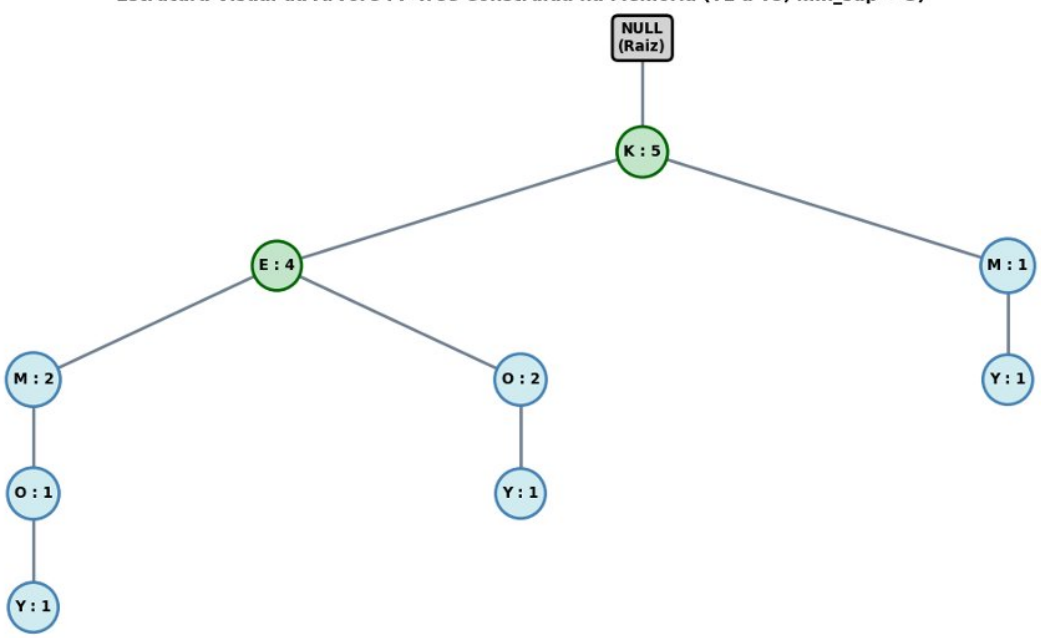
```
1. Lista Ordenada de Itens Frequentes (L): [('K', 5), ('E', 4), ('M', 3), ('O',
3), ('Y', 3)]

2. Transações Filtradas e Reordenadas Inseridas na FP-Tree:
   T1: Bruta=['E', 'K', 'M', 'N', 'O', 'Y'] -> Ordenada=['K', 'E', 'M', 'O',
'Y']
   T2: Bruta=['D', 'E', 'K', 'N', 'O', 'Y'] -> Ordenada=['K', 'E', 'O', 'Y']
   T3: Bruta=['A', 'E', 'K', 'M'] -> Ordenada=['K', 'E', 'M']
   T4: Bruta=['C', 'K', 'M', 'U', 'Y'] -> Ordenada=['K', 'M', 'Y']
```

```
   T5: Bruta=['C', 'E', 'I', 'K', 'O'] -> Ordenada=['K', 'E', 'O']

3. Mineração Bottom-Up (Base de Padrões Condicionais e Árvore FP Condicional):
   Item 'Y': Base Condicional = [(['K', 'E', 'M', 'O'], 1), (['K', 'E', 'O'],
1), (['K', 'M'], 1)]
            -> Árvore FP Condicional (>= 3) = {'K': 3} | Pares = [({'Y', 'K'},
3)]
   Item 'O': Base Condicional = [(['K', 'E', 'M'], 1), (['K', 'E'], 2)]
            -> Árvore FP Condicional (>= 3) = {'E': 3, 'K': 3} | Pares = [({'E',
'O'}, 3), ({'O', 'K'}, 3)]
   Item 'M': Base Condicional = [(['K', 'E'], 2), (['K'], 1)]
            -> Árvore FP Condicional (>= 3) = {'K': 3} | Pares = [({'M', 'K'},
3)]
   Item 'E': Base Condicional = [(['K'], 4)]
            -> Árvore FP Condicional (>= 3) = {'K': 4} | Pares = [({'E', 'K'},
4)]
   Item 'K': Base Condicional = []
            -> Árvore FP Condicional (>= 3) = {} | Pares = []
```

Estrutura Visual da Árvore FP-Tree Construída na Memória (T1 a T5, min_sup = 3)



1.6 Experimento 5: Comparação de Desempenho Real (`import time`) – Apriori vs. FP-Growth sob Diferentes Limiares de Suporte

[11]:
```
np.random.seed(12)
n_compras = 2000
n_produtos = 30
nomes_produtos = [f"P{i:02d}" for i in range(n_produtos)]
prob_produtos = np.linspace(0.42, 0.08, n_produtos)
matriz_compras_bin = (np.random.rand(n_compras, n_produtos) < prob_produtos).
 ↪astype(int)
lista_compras = [[nomes_produtos[j] for j in range(n_produtos) if␣
 ↪matriz_compras_bin[i, j] == 1]
                 for i in range(n_compras)]

def executar_apriori_completo(transacoes_set, itens_unicos, min_contagem):
    L_atual = {(i,): sum(1 for t in transacoes_set if i in t) for i in␣
 ↪itens_unicos}
    L_atual = {k: v for k, v in L_atual.items() if v >= min_contagem}
    todos_frequentes = dict(L_atual)
    k = 2
    while len(L_atual) >= 2:
        itens_ativos = sorted(list(set(x for tupla in L_atual.keys() for x in␣
 ↪tupla)))
        C_k = [comb for comb in combinations(itens_ativos, k)
               if all(sub in L_atual for sub in combinations(comb, k - 1))]
        if not C_k:
            break
        proximo_L = {}
        for cand in C_k:
            s_cand = set(cand)
            cont = sum(1 for t in transacoes_set if s_cand.issubset(t))
            if cont >= min_contagem:
                proximo_L[cand] = cont
        todos_frequentes.update(proximo_L)
        L_atual = proximo_L
        k += 1
    return todos_frequentes

def executar_fpgrowth_completo(transacoes_lista, min_contagem):
    padroes_encontrados = {}

    def minerar_recursivo(transacoes_ponderadas, sufixo_atual):
        freq = defaultdict(int)
        for t, peso in transacoes_ponderadas:
            for item in t:
                freq[item] += peso
        freq = {k: v for k, v in freq.items() if v >= min_contagem}
```

```
        if not freq:
            return

        ordem = sorted(freq.keys(), key=lambda x: (-freq[x], x))
        raiz = NoFPTree("NULL", 0, None)
        cabecalho = {item: [] for item in ordem}

        for t, peso in transacoes_ponderadas:
            t_ord = [i for i in ordem if i in t]
            atual = raiz
            for item in t_ord:
                if item in atual.filhos:
                    atual.filhos[item].contagem += peso
                else:
                    novo = NoFPTree(item, peso, atual)
                    atual.filhos[item] = novo
                    cabecalho[item].append(novo)
                atual = atual.filhos[item]

        for item in reversed(ordem):
            novo_padrao = tuple(sorted(sufixo_atual + [item]))
            padroes_encontrados[novo_padrao] = freq[item]
            base_condicional = []
            for no in cabecalho[item]:
                caminho = []
                pai = no.pai
                while pai is not None and pai.item != "NULL":
                    caminho.append(pai.item)
                    pai = pai.pai
                if caminho:
                    base_condicional.append((caminho, no.contagem))
            if base_condicional:
                minerar_recursivo(base_condicional, list(novo_padrao))

    trans_iniciais = [(t, 1) for t in transacoes_lista]
    minerar_recursivo(trans_iniciais, [])
    return padroes_encontrados

transacoes_set = [set(t) for t in lista_compras]
suportes_teste = [0.25, 0.20, 0.16, 0.12, 0.09]
tempos_apriori_ms = []
tempos_fpgrowth_ms = []
qtd_padroes = []
print("=== BENCHMARK DE TEMPO REAL (import time): APRIORI vs. FP-GROWTH (N =␣
  ↪2000 Transações) ===")
for sup in suportes_teste:
    min_cont = int(sup * n_compras)
```

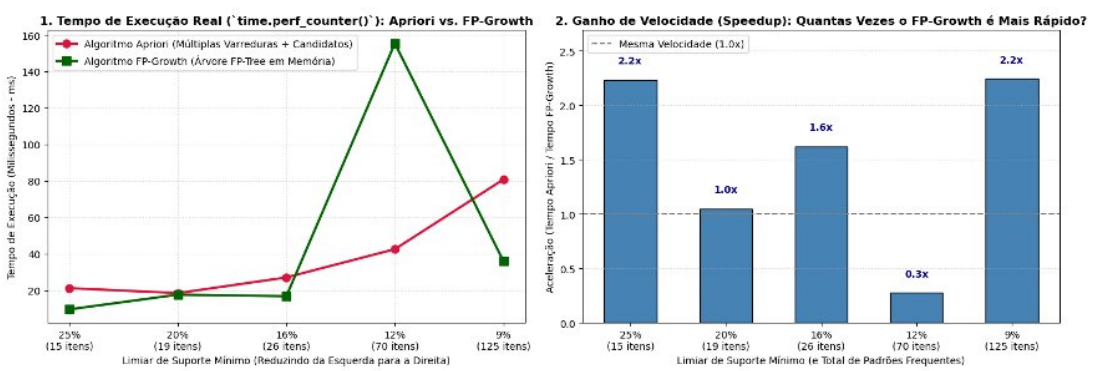
```
    t0_ap = time.perf_counter()
    freq_ap = executar_apriori_completo(transacoes_set, nomes_produtos,␣
 ↪min_cont)
    t1_ap = time.perf_counter()
    tempo_ap = (t1_ap - t0_ap) * 1000.0
    t0_fp = time.perf_counter()
    freq_fp = executar_fpgrowth_completo(lista_compras, min_cont)
    t1_fp = time.perf_counter()
    tempo_fp = (t1_fp - t0_fp) * 1000.0
    tempos_apriori_ms.append(tempo_ap)
    tempos_fpgrowth_ms.append(tempo_fp)
    qtd_padroes.append(len(freq_fp))
    speedup = tempo_ap / tempo_fp if tempo_fp > 0 else 1.0
    print(f"Suporte Mín = {sup*100:4.1f}% | Padrões Encontrados: {len(freq_fp):
 ↪3d} (Iguais: {len(freq_ap)==len(freq_fp)}) "
          f"| Apriori: {tempo_ap:7.2f} ms | FP-Growth: {tempo_fp:6.2f} ms ->␣
 ↪Speedup: {speedup:.1f}x mais rápido!")
fig, (ax_tempo, ax_speed) = plt.subplots(1, 2, figsize=(15, 5.2))
rotulos_sup = [f"{int(s*100)}%\n({q} itens)" for s, q in zip(suportes_teste,␣
 ↪qtd_padroes)]
ax_tempo.plot(rotulos_sup, tempos_apriori_ms, marker='o', color='crimson', lw=2.
 ↪5, markersize=8,
              label='Algoritmo Apriori (Múltiplas Varreduras + Candidatos)')
ax_tempo.plot(rotulos_sup, tempos_fpgrowth_ms, marker='s', color='darkgreen',␣
 ↪lw=2.5, markersize=8,
              label='Algoritmo FP-Growth (Árvore FP-Tree em Memória)')
ax_tempo.set_title("1. Tempo de Execução Real (`time.perf_counter()`): Apriori␣
 ↪vs. FP-Growth", fontweight='bold')
ax_tempo.set_xlabel("Limiar de Suporte Mínimo (Reduzindo da Esquerda para a␣
 ↪Direita)")
ax_tempo.set_ylabel("Tempo de Execução (Milissegundos - ms)")
ax_tempo.legend(loc='upper left')
ax_tempo.grid(True, linestyle=':', alpha=0.6)

speedups = [ta / tf for ta, tf in zip(tempos_apriori_ms, tempos_fpgrowth_ms)]
barras = ax_speed.bar(rotulos_sup, speedups, color='steelblue', edgecolor='k',␣
 ↪width=0.55)
ax_speed.axhline(y=1.0, color='gray', linestyle='--', label='Mesma Velocidade␣
 ↪(1.0x)')
for barra, valor_sp in zip(barras, speedups):
    ax_speed.text(barra.get_x() + barra.get_width()/2, barra.get_height() + 0.
 ↪15,
                  f"{valor_sp:.1f}x", ha='center', fontweight='bold',␣
 ↪color='navy')
```

```
ax_speed.set_title("2. Ganho de Velocidade (Speedup): Quantas Vezes o FP-Growth␣
 ↪é Mais Rápido?", fontweight='bold')
ax_speed.set_xlabel("Limiar de Suporte Mínimo (e Total de Padrões Frequentes)")
ax_speed.set_ylabel("Aceleração (Tempo Apriori / Tempo FP-Growth)")
ax_speed.set_ylim(0, max(speedups) * 1.2)
ax_speed.legend(loc='upper left')
ax_speed.grid(True, axis='y', linestyle=':', alpha=0.6)

plt.tight_layout()
plt.show()
```

```
=== BENCHMARK DE TEMPO REAL (import time): APRIORI vs. FP-GROWTH (N = 2000
Transações) ===
Suporte Mín = 25.0% | Padrões Encontrados:  15 (Iguais: True) | Apriori:   21.09
ms | FP-Growth:   9.44 ms -> Speedup: 2.2x mais rápido!
Suporte Mín = 20.0% | Padrões Encontrados:  19 (Iguais: True) | Apriori:   18.35
ms | FP-Growth:  17.51 ms -> Speedup: 1.0x mais rápido!
Suporte Mín = 16.0% | Padrões Encontrados:  26 (Iguais: True) | Apriori:   26.96
ms | FP-Growth:  16.65 ms -> Speedup: 1.6x mais rápido!
Suporte Mín = 12.0% | Padrões Encontrados:  70 (Iguais: True) | Apriori:   42.45
ms | FP-Growth: 155.33 ms -> Speedup: 0.3x mais rápido!
Suporte Mín =  9.0% | Padrões Encontrados: 125 (Iguais: True) | Apriori:   80.67
ms | FP-Growth:  35.99 ms -> Speedup: 2.2x mais rápido!
```



[ ]: